

Backend Fundamentals — Recap 1

What is an API, and how does Express routing actually work?

1. What is an API?

Think of a restaurant. You (the customer) don't walk into the kitchen and cook your own food. You sit at a table, and a **waiter** takes your order, brings it to the kitchen, and brings the food back to you. You never touch the kitchen directly — you just interact with the waiter using a menu (a fixed set of things you're allowed to ask for).

An **API** is that waiter.

- Your **frontend** (a website, an app, or just you typing a URL) is the customer.
- Your **backend server** is the kitchen — where the real work happens.
- The **API** is the set of rules for how the customer is allowed to talk to the kitchen.

In short: building an API means writing a program that sits and waits for requests, then sends back responses.

2. The Request / Response Cycle

Every time you visit a website, your browser sends a **request** to a server, and the server sends back a **response**. Visiting a URL normally sends a request using the **GET** method — meaning "give me something." Other actions, like saving new data, use different methods (e.g. POST), which a plain browser address bar can't send on its own — that's why testing tools or commands like curl exist.

3. Who Actually Creates the Server: Node or Express?

Node.js creates and hosts the server. Node has a built-in module called **http** that can create a server all on its own, with no extra libraries:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Hello!');
});

server.listen(3000);
```

Express is built on top of this. When you write `app.listen(3000, ...)`, Express is internally doing that same job for you — it just wraps it in a friendlier interface so you don't have to manually parse URLs or match routes by hand.

Mental model: **Node = the actual server. Express = the convenient toolkit built on top of it.**

4. Routing: URL + Method → Function

A route has two parts: an **address** (the URL, like `/hello`) and a **verb** (the HTTP method — GET, POST, PUT, DELETE — describing the action). The REST-style convention is to put the **thing** in the URL, and let the method describe the action:

What you want to do	URL	Method
See all tasks	<code>/tasks</code>	GET
See one task	<code>/tasks/5</code>	GET
Create a task	<code>/tasks</code>	POST
Update a task	<code>/tasks/5</code>	PUT
Delete a task	<code>/tasks/5</code>	DELETE

Example route in Express:

```
app.get('/hello', (req, res) => {  
  res.send('Hello, this is my first API!');  
});
```

5. Node Does Not Auto-Reload

Node.js reads your file **once**, the moment you run `node index.js`, and keeps running that exact version in memory. If you edit the file afterward — even just adding a route — the running server has no idea. You must stop it (Ctrl+C) and run `node index.js` again for changes to take effect.

(Coming later: a tool called **nodemon** that restarts the server automatically on save.)

6. The "Cannot GET /x" Error

This message comes from **Express itself**, not the browser. It means: a request reached your server successfully, but Express checked its list of defined routes and found no match for that exact URL + method combination. Express only responds to routes you've explicitly written — nothing happens automatically for undefined URLs; you just get this default error instead.

7. Key Mental Model To Keep

- A browser visiting a URL always sends a GET request automatically.
- Node.js creates the server; Express organizes routing on top of it.
- Every route = a URL + an HTTP method, mapped to a function.
- No matching route → Express replies with "Cannot GET /path" by default.
- Code changes need a server restart to take effect — Node doesn't watch files on its own.

Next up: how POST actually works — sending data to the server, not just asking for it.